

Okay, here's a detailed design document outlining the ASV Simulator project.

ASV Simulator Design Document

1. Overview

This document describes the design of a Python-based simulator for an Autonomous Surface Vehicle (ASV) tasked with navigating a pre-qualification course. The course involves passing through a gate, maneuvering around a marker pole, and returning through the gate.

The simulator provides a 2D top-down map view and a simulated 3D first-person camera view. It models an ASV operating on the surface ($Z=0$) using a differential drive (two thrusters). The AI employs a modular Task/Subtask architecture for mission execution and relies on simulated computer vision (OpenCV blob detection) for navigation.

2. Key Program Concepts

- **Task/Subtask Architecture:** The core AI logic is structured hierarchically:
 - **Tasks** (`ai.tasks.task_base.Task`): Represent high-level mission goals (e.g., GateTask, MarkerTurnTask). Each task manages a sequence of subtasks and often performs specific vision processing related to its goal.
 - **Subtasks** (`ai.tasks.subtask_base.Subtask`): Represent reusable, atomic actions (e.g., TurnToHeading, DriveStraight, AlignToObjectX). They are executed sequentially by the parent Task.
- **Context Dictionary:** A shared dictionary (`Task.context`) is used for indirect communication between subtasks within a Task. For instance, an alignment subtask stores the final heading, which a subsequent turning subtask can retrieve to calculate a relative turn.
- **Physics Model:** (`simulator.py:applyPhysics`): Simulates a simplified 2D rigid body model for the ASV.
 - **Differential Drive:** Calculates surge (forward/backward) force and yaw (turning) torque based on independent port and starboard thruster commands.

- **Drag:** Implements distinct quadratic drag coefficients for surge (surgeDragCoeff) and sway (swayDragCoeff, typically much higher for a boat hull), plus angular drag (angularDragCoeff) resisting rotation.
- **Inertia:** Includes mass (subMass) and rotational inertia (subInertia) affecting linear and angular acceleration.
- **Surface Constraint:** The boat's Z-position and pitch angle are locked to zero.
- **Simulated Vision:** (ai.vision.find_blobs_hsv, Task process_vision methods):
 - The find_blobs_hsv function uses OpenCV (cv2) to find contiguous areas (blobs) in the simulated camera image that fall within specified HSV color ranges (defined in config.py).
 - Individual Task classes override the process_vision method to interpret these raw blobs, identify specific targets (like the gate pole pair or the marker pole), and populate a VisionData object with relevant information (e.g., target visibility, center coordinates, apparent size) for use by subtasks.
- **Configuration:** (config.py): Centralizes constants, tuning parameters (PID gains, drag coefficients, AI thresholds), color definitions (RGB and HSV), and course layout dimensions.

3. Program Structure

- **main.py:**
 - The main entry point for the application.
 - Defines the mission plan as an ordered list of Task objects.
 - Instantiates the Submarine (AI controller) and the SubmarineSimulator.
 - Starts the simulation loop (sim.run()).
- **simulator.py:**
 - Contains the SubmarineSimulator class.
 - Initializes Pygame and creates the display window.
 - Loads assets (e.g., camera background image).

- Manages the main simulation loop (run method):
 - Handles user input (handleInput).
 - Updates the AI (submarineAI.update).
 - Applies physics based on thruster commands (applyPhysics).
 - Generates the 3D camera view (generateCameraView, project3D).
 - Renders the 2D map and UI elements (render, _renderUi).
- Defines physics parameters (mass, inertia, drag coefficients).
- Creates the course elements (resetSimulation).
- **config.py:**
 - Defines color constants (RGB tuples).
 - Defines HSV color range tuples/lists for vision detection.
 - Contains dataclasses (SimulationConfig, PrequalConfig) holding world dimensions, camera FOV, boat dimensions, course layout, physics constants, and AI tuning gains.
- **world.py:**
 - Defines dataclasses representing physical objects (PrequalGate, PrequalMarker).
 - Defines SubmarinePhysicsState dataclass to hold the boat's position, orientation, and velocities.
- **data_structures.py:**
 - Defines simple dataclasses for data transfer:
 - ThrusterCommands: Holds port/starboard thrust values (-1 to 1).
 - SensorSuite: Bundles all simulated sensor readings (camera image, heading, IMU, etc.).
 - MPU6050Readings: Simulated IMU data (accelerations, gyro rates).
 - VisionData: Standard structure for tasks to report vision results (target visibility, position, size).

- **utils.py:** Contains helper functions (e.g., `angle_diff` for calculating the shortest angle between two headings).
- **ai/:**
 - **submarine.py:**
 - Contains the main Submarine AI class.
 - `__init__`: Stores the mission plan, initializes AI gains.
 - `update`: Executes the current Task, handles task transitions (COMPLETED) or failures (FAILED).
 - `reset`: Resets the mission state.
 - Low-level control methods (called by Tasks/Subtasks):
 - `_mix_and_normalize_commands`: Converts surge/sway/yaw requests into port/starboard commands.
 - `get_heading_commands`: Generates commands to turn towards/maintain a heading.
 - `_get_pid_hover_commands`: Generates commands to hold a specific X, Y position using PID control.
 - `_get_damping_commands`: Generates commands to resist current motion.
 - `get_search_commands`: Generates commands for spinning search patterns.
 - `get_go_to_visual_target_commands`: Generates commands to steer towards a pixel coordinate in the camera view.
 - **vision.py:** Contains `find_blobs_hsv` using OpenCV for color segmentation and contour finding.
 - **tasks/:**
 - **task_base.py:** Defines Task base class (manages subtask list, `current_subtask_index`, `context`) and `TaskStatus` enum. Its `execute` method runs `process_vision` and the current subtask, handling subtask completion/failure and context passing.
 - **subtask_base.py:** Defines Subtask base class (defines `on_enter`, `execute`, `on_exit` interface) and `SubtaskStatus` enum.

- **common_subtasks.py:** Implements reusable Subtasks (TurnToHeading, DriveStraight, Stabilize, WaitForTargetVisible, AlignToObjectX, ApproachTargetVisualWidth, DriveUntilTargetLost).
- **gate_task.py:** Task for gate navigation. Overrides process_vision to find pole pairs. Defines a subtask sequence using common subtasks.
- **marker_turn_task.py:** Task for marker navigation. Overrides process_vision to find the green pole. Includes custom inline subtasks (_StoreHeadingSubtask, _OrbitPoleSubtask). Overrides execute to handle subtask failures by restarting itself.
- **stabilize_task.py:** Simple Task wrapper around the Stabilize subtask (potentially redundant).

4. Transitioning to Hardware

Moving the AI from this simulator to a physical ASV involves replacing simulated components with real-world interfaces and extensive tuning:

1. Hardware Platform:

- **Compute:** A microcontroller or single-board computer (like Raspberry Pi, Jetson Nano) running Linux/ROS or a real-time OS.
- **Sensors:**
 - **Camera:** USB or CSI camera connected to the compute board.
 - **IMU:** Inertial Measurement Unit (e.g., MPU6050, BNO055) connected via I2C/SPI for heading (requires sensor fusion like a Kalman filter for reliable heading), pitch/roll (less critical for surface boat but useful), and angular rates.
 - **GPS:** For absolute positioning (optional for this course but useful for larger areas).
 - **Depth Sensor:** Not needed for ASV.
- **Actuators:**
 - Two **Brushless DC Motors** with propellers (e.g., T200 thrusters).

- **Electronic Speed Controllers (ESCs):** To drive the motors based on signals from the compute board.
- **Power:** Batteries, power distribution, voltage regulators.
- **Communication:** (Optional) Wi-Fi or radio telemetry for monitoring/control.

2. Software Adaptation:

- **Replace SensorSuite:** Instead of reading from SubmarinePhysicsState, read data from actual sensor drivers/libraries (e.g., smbus2 for I2C IMU, OpenCV VideoCapture for camera, GPS library). Implement sensor fusion for heading.
- **Replace Simulated Vision:** The find_blobs_hsv function in ai/vision.py will work directly on real camera frames captured via OpenCV. **Crucially, the HSV ranges in config.py will need significant retuning** based on real-world lighting, water conditions, and camera characteristics.
- **Replace ThrusterCommands Output:** Instead of the simulator consuming ThrusterCommands, the AI needs to output appropriate signals (e.g., PWM duty cycles via GPIO pins) compatible with the chosen ESCs. This requires mapping the -1 to 1 thrust values to the ESC's signal range (e.g., 1100 μ s to 1900 μ s pulse width).
- **Replace applyPhysics:** The real world handles physics! Remove the simulator's physics update.

3. Tuning:

- **PID Gains:** All PID gains (HOVER_XY_P/I/D, HOVER_YAW_P, ALIGN_YAW_P, YAW_D, etc. in ai/submarine.py) will need extensive re-tuning on the actual boat to match its mass, inertia, thruster power, and hydrodynamic properties. Start with low gains and increase gradually.
- **Vision Thresholds:** HSV ranges, MIN_PIXELS_FOR_DETECTION, and potentially subtask parameters like APPROACH_WIDTH_THRESHOLD_PX will need tuning for reliable detection.
- **Subtask Parameters:** Durations (DriveStraight, Stabilize), surge power levels (ORBIT_SURGE_POWER), and completion tolerances (TURN_COMPLETION_TOLERANCE_DEG) will need adjustment based on the boat's real-world performance.

4. Real-World Considerations:

- **Latency:** Sensor readings and command execution take time. Control loops must account for this.
- **Noise:** Real sensor data is noisy. Filtering (e.g., moving averages, Kalman filters) might be necessary.
- **Calibration:** IMU needs calibration. Camera needs color calibration (HSV tuning). ESCs need PWM range calibration.
- **Power Management:** Monitor battery levels.
- **Environmental Factors:** Wind, waves, and currents will affect the boat's motion and need to be accounted for, potentially requiring more robust controllers.

The core Task/Subtask logic should be largely transferable, but the interfaces to the physical world (sensors and actuators) and the tuning parameters will require significant effort.

Okay, here's a section detailing how the GateTask uses its subtasks:

5. Task and Subtask Details

This section details how the primary Task implementations use their sequence of Subtasks to achieve their objectives.

5.1 GateTask - Navigating the Gate

The GateTask is responsible for finding the gate (identified by two gray poles), aligning with it, and driving through it. It uses the following sequence of subtasks:

1. **WaitForTargetVisible():**

- **Goal:** Find the gate visually.

- **Action:** This subtask runs until the `GateTask.process_vision` method successfully identifies the pair of gray poles and sets `gate_vision_data.gate_is_visible = True`.
- **Control:** If the gate isn't visible, this subtask returns `RUNNING` with zero commands. The main `Task.execute` logic detects this and commands the boat to spin (using `DEFAULT_SEARCH_TURN_POWER` and `search_direction`). Once the gate *is* visible, the subtask returns `COMPLETED` and commands damping to stop the spin.

2. **AlignToObjectX(target_x_fraction=0.5, ...):**

- **Goal:** Point the boat directly at the center of the gate opening.
- **Action:** This subtask takes the `align_target_center_x` value (calculated by `GateTask.process_vision` as the midpoint between the two detected poles) and commands yaw adjustments to bring this pixel coordinate to the horizontal center (0.5) of the camera view. It uses a Proportional-Derivative (PD) controller based on pixel error and gyro rate.
- **Control:** It commands only yaw (pivot turn, zero surge). It returns `COMPLETED` when the target is centered within `tolerance_px` and the boat's rotation rate (`gyro_z`) is below `yaw_rate_tolerance`. Upon completion, it stores the boat's current heading in `context['initial_heading']`.

3. **Stabilize(duration=1.0, ...):**

- **Goal:** Stop any residual motion (especially sway or yaw) after alignment before driving forward.
- **Action:** This subtask uses the boat's PID position controller (`_get_pid_hover_commands`) to actively hold the boat at the position and heading it had when the subtask started.
- **Control:** It runs for a fixed duration (1.0 second) and completes if the boat's speed is below `speed_threshold`.

4. **DriveUntilTargetLost(surge_power=0.6):**

- **Goal:** Drive straight through the gate opening.
- **Action:** This subtask retrieves the `initial_heading` stored in the context by `AlignToObjectX` (or uses the heading locked by `Stabilize`). It commands the boat to drive straight forward (`surge_power=0.6`) while maintaining that heading.
- **Control:** It continuously checks if `GateTask.process_vision` still reports `gate_is_visible = True`. As long as the gate is visible, it returns `RUNNING`.

Once `gate_is_visible` becomes `False` (meaning the poles have likely passed out of view), it returns `COMPLETED`. It uses the `target_was_visible` flag internally to ensure it only completes *after* having seen the gate at least once during its execution.

5. **DriveStraight(duration=4.0, ...):**

- **Goal:** Ensure the boat has fully cleared the physical gate structure after losing visual contact.
- **Action:** This subtask continues driving straight on the *same heading* established by the previous subtasks.
- **Control:** It runs for a fixed duration (4.0 seconds) and then returns `COMPLETED`.

6. **Stabilize(duration=1.0):**

- **Goal:** Stop the boat after successfully clearing the gate.
- **Action:** Same as subtask 3, uses PID control to halt motion.
- **Control:** Runs for a fixed duration and checks speed threshold before returning `COMPLETED`.

Once the final `Stabilize` subtask completes, the base `Task.execute` logic recognizes it's the end of the subtask list and returns `TaskStatus.COMPLETED` to the main `Submarine.update` loop, allowing the mission to proceed to the next `Task`.

Okay, here's section 5.2 detailing the `MarkerTurnTask`.

5.2 **MarkerTurnTask - Maneuvering Around the Marker**

The `MarkerTurnTask` is responsible for finding the green marker pole, navigating around it in a controlled arc (orbit), and turning to face the opposite direction (back towards the gate). It uses the following sequence of subtasks:

1. **WaitForTargetVisible():**

- **Goal:** Find the green marker pole visually.

- **Action:** This subtask runs until the `MarkerTurnTask.process_vision` method successfully identifies the green pole and sets `marker_vision_data.gate_is_visible = True` (using the flag generically).
- **Control:** If the marker isn't visible, this subtask returns `RUNNING` with zero commands. The main `Task.execute` logic detects this and commands the boat to spin right (`search_direction=-1`, set during task restart on failure) or left (default). Once the marker *is* visible, the subtask returns `COMPLETED` and commands damping to stop the spin.

2. `AlignToObjectX(target_x_fraction=0.60, ..., yaw_gain=3.0, ...)`:

- **Goal:** Point the boat on a path slightly offset from the marker, preparing for the orbit.
- **Action:** Takes the `align_target_center_x` (center of the green pole from `MarkerTurnTask.process_vision`) and commands yaw adjustments to bring this pixel coordinate to the 60% mark (0.60) from the left edge of the camera view. This places the pole slightly right-of-center. A high `yaw_gain` (3.0) is used for assertive control.
- **Control:** Commands only yaw (pivot turn). Returns `COMPLETED` when the target is at the offset position within `tolerance_px` and the boat's rotation rate is below `yaw_rate_tolerance`. Upon completion, it stores the boat's current heading in `context['initial_heading']`.

3. `_StoreHeadingSubtask()`:

- **Goal:** Capture the precise heading established by the offset alignment. (Note: `AlignToObjectX` now does this in its `on_exit`). This subtask became redundant but remains in the list for clarity of the original design intent - it essentially does nothing now as `AlignToObjectX` handles storing the heading in the context.
- **Action:** Reads the current `sensors.heading` and stores it in `context['initial_heading']`.
- **Control:** Completes immediately (`SubtaskStatus.COMPLETED`).

4. `_OrbitPoleSubtask()`:

- **Goal:** Drive the boat in a controlled arc around the pole until a significant heading change is achieved.
- **Action:** Retrieves the `initial_heading` from the context. It commands a constant forward `ORBIT_SURGE_POWER` (0.3). Simultaneously, it uses yaw control (with aggressive `ORBIT_YAW_GAIN = 3.0`) to actively keep

the green pole visually positioned at the ORBIT_TARGET_X_FRACTION (60%) mark on the screen.

- **Control:** Returns RUNNING while orbiting. If the pole is lost, it initiates a local rightward search (local_search_turn_power = -0.25, zero surge) for lost_search_timeout (3.0 seconds). If the pole is reacquired, it resumes orbiting. If the timeout expires, it returns FAILED. The subtask returns COMPLETED once the boat's current heading has changed by more than ORBIT_COMPLETION_ANGLE_DEG (110 degrees) relative to the initial_heading. Upon completion, it calculates the final target heading (180 degrees from initial_heading) and stores it in context['final_turn_target'].

5. TurnToHeading(relative_degrees=-180.0, ...):

- **Goal:** Turn the boat to face the return direction (back towards the gate).
- **Action:** On entering, it reads the initial_heading from the context and calculates the absolute target heading required for a -180 degree (right turn) relative change.
- **Control:** Commands only yaw (pivot turn) towards the calculated absolute target heading. Returns COMPLETED when the boat's heading is within tolerance_degrees of the target.

Once the final TurnToHeading subtask completes, the base Task.execute logic returns TaskStatus.COMPLETED to Submarine.update, and the mission proceeds. If the _OrbitPoleSubtask fails after its local search, MarkerTurnTask.execute catches the FAILED status, calls self.reset(search_direction=-1), and returns RUNNING, effectively restarting the task with a rightward search bias.